

Multi-Domain Real-Time Object Tracking and Telemetry Analytics Engine using Fine-Tuned YOLOv8 and ByteTrack

[Raihan Mahmud]*, [Md Emran Hossain]*, [Dewan Mim]*, [Fahim Muntasir Siam]*

*Department of Electrical and Computer Engineering, North South University, Dhaka, Bangladesh

CSE445: Machine Learning (Section 7), Group 07

Faculty: Dr. Mohammad Shifat-E-Rabbi [MSRb]

IDs: [2222642642], [2221132642], [2122416], [2221277]

Email: {[raihan.mahmud], [emran.hossain2], [dewan.mim], [Fahim.muntasir]}@northsouth.edu

Abstract—Multi-object tracking across diverse video domains—such as aerial surveillance, traffic monitoring, and sports analytics—suffers from severe performance drops when using single-domain models due to scale variations and occlusions. Furthermore, existing pipelines rarely address the engineering challenge of rendering heterogeneous telemetry and incompatible video codecs on web interfaces. This work presents a unified tracking and analytics engine combining a fine-tuned YOLOv8 detector with a ByteTrack association layer to ensure identity persistence under high occlusion. Additionally, we introduce a server-side FFmpeg transcoding module for browser video compatibility and a Dynamic Data Adapter that normalizes inconsistent CSV telemetry schemas for a Next.js dashboard. Our fine-tuned YOLOv8 achieves an mAP@50 of [XX.X]%, running at [XX.X] FPS on [GPU model]. Code and model weights are publicly available on GitHub and Hugging Face.

Index Terms—Computer Vision, Real-Time Object Tracking, YOLOv8, ByteTrack, Kalman Filtering, Hungarian Algorithm, Telemetry Data Normalization, Next.js, H.264 Video Transcoding, Multi-Object Tracking.

I. INTRODUCTION

A. Background

Multi-object tracking (MOT) is a core computer vision task that maintains temporally consistent identities for multiple objects across video sequences. The canonical tracking-by-detection pipeline consists of two coupled subproblems: (i) per-frame object detection, which localizes candidate objects via bounding boxes, and (ii) inter-frame data association, which links detections across consecutive frames into trajectories.

Despite substantial advances, MOT systems remain sensitive to domain shift. Trackers tuned for ground-level surveillance fail to translate directly to aerial, drone, crowd, or sports footage, where targets exhibit small scales, rapid ego-motion, high object density, and frequent occlusions that violate classical constant-velocity assumptions.

II. PROBLEM STATEMENT

Extracting continuous trajectory telemetry from high-density video feeds is challenging due to frequent occlusions,

rapid scale changes, and trajectory overlaps that trigger identity switches (IDSW) in conventional Multi-Object Tracking (MOT) pipelines. Additionally, high-accuracy trackers often sacrifice real-time throughput (> 30 FPS), limiting their deployment in smart traffic and sports analytical systems. This work addresses these limitations by presenting an end-to-end framework that pairs fine-tuned YOLOv8 object detection with ByteTrack data association. By retaining low-score bounding boxes during Kalman state prediction and Hungarian matching, the proposed engine maintains track continuity across heavy occlusions while generating structured spatial analytics in real time.

A. Research Contributions

The main contributions of this work are:

- 1) **Multi-Domain YOLOv8 Detector:** Fine-tuned YOLOv8 on a multi-domain dataset (aerial, traffic, pedestrian, sports) to generalize across small-scale and ground-level objects using a single detection backbone.
- 2) **ByteTrack Association under Occlusion:** Integrated ByteTrack with a two-stage Hungarian matching algorithm and an 8-dimensional Kalman filter to minimize identity switches during high-occlusion sequences.
- 3) **Transcoding and Normalization Pipeline:** Implemented a server-side FFmpeg transcoding module and a Dynamic Data Adapter to convert arbitrary video codecs and heterogeneous CSV schemas into a unified format for a Next.js dashboard.
- 4) **Open-Source Release:** Published model weights on Hugging Face and source code on GitHub to ensure reproducibility.

The remainder of this paper is organized as follows: Section II reviews related work; Section III outlines mathematical foundations; Section IV details system architecture; Section V presents experimental results; and Section VI concludes the study.

III. RELATED WORK

A. Deep Learning Object Detectors

Object detection paradigms are divided into two-stage (e.g., Faster R-CNN [?]) and single-stage detectors. While two-stage models provide high localization accuracy via region proposals, their sequential pipeline introduces significant inference latency. In contrast, single-stage detectors like the YOLO family formulate detection as a direct dense regression problem. YOLOv8—the architecture adopted in this work—features an anchor-free detection head and a C2f module that improves gradient flow while reducing parameter count, making it well-suited for multi-domain and aerial deployment.

B. Multi-Object Tracking Algorithms

Tracking-by-detection algorithms have evolved to handle object identity association. SORT [3] combines a Kalman filter with the Hungarian algorithm using Intersection-over-Union (IoU) matching, but suffers from identity switches under occlusion due to high confidence thresholds. DeepSORT [4] adds appearance embeddings (Re-ID) to mitigate this at the cost of increased latency. ByteTrack [5], utilized in this work, performs a two-stage association by recovering low-confidence detections, effectively preserving track IDs under occlusion without the heavy computational overhead of appearance networks.

C. Web-Based Machine Learning Deployment Pipelines

Deploying multi-object tracking pipelines for web consumption presents systems-level challenges. Raw video files often use codecs (e.g., H.265/HEVC, VP9) that lack universal browser support, requiring server-side FFmpeg transcoding to standard H.264 (AVC). Additionally, heterogeneous CSV telemetry schemas across different domains require programmatic normalization. Leveraging modern frameworks like Next.js enables unified full-stack dashboard rendering, seamlessly integrating video playback with synchronized telemetry.

IV. MATHEMATICAL FORMULATION AND METHODOLOGY

This section formalizes the loss functions governing detector fine-tuning, the motion model underlying track state prediction, and the combinatorial optimization formulation underlying data association.

A. YOLOv8 Detection Loss Function

YOLOv8 fine-tuning minimizes a composite loss $\mathcal{L}_{\text{total}}$ comprising a bounding box regression term, a classification term, and a distribution-based localization refinement term:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{box}} \mathcal{L}_{\text{CIoU}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{diff}} \mathcal{L}_{\text{DFL}} \quad (1)$$

where λ_{box} , λ_{cls} , and λ_{diff} are empirically tuned weighting coefficients (set to [7.5, 0.5, 1.5] respectively in our fine-tuning configuration, consistent with Ultralytics default hyperparameters).

1) *Complete IoU (CIoU) Loss*: Standard IoU loss suffers from vanishing gradients when predicted and ground-truth boxes are disjoint. Complete IoU (CIoU) loss addresses this by jointly penalizing overlap area, central point distance, and aspect ratio consistency:

$$\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v \quad (2)$$

where $\text{IoU} = \frac{|\mathbf{B} \cap \mathbf{B}^{gt}|}{|\mathbf{B} \cup \mathbf{B}^{gt}|}$ is the standard intersection-over-union between predicted box $\mathbf{B}$ and ground-truth box $\mathbf{B}^{gt}$; $\rho(\mathbf{b}, \mathbf{b}^{gt})$ denotes the Euclidean distance between the predicted and ground-truth box center points $\mathbf{b}$ and $\mathbf{b}^{gt}$; c is the diagonal length of the smallest enclosing box covering both $\mathbf{B}$ and $\mathbf{B}^{gt}$; and v measures aspect ratio consistency:

$$v = \frac{4}{\pi^2} \left(\arctan \frac{w^{gt}}{h^{gt}} - \arctan \frac{w}{h} \right)^2 \quad (3)$$

with the trade-off parameter α defined as:

$$\alpha = \frac{v}{(1 - \text{IoU}) + v} \quad (4)$$

The $\rho^2(\cdot)/c^2$ term is of particular practical relevance to our aerial/drone fine-tuning subset, where ground-truth boxes are small and sparsely distributed; this term provides a meaningful gradient signal even when predicted and ground-truth boxes have zero IoU overlap during early training, a common occurrence for small-object detection at high resolution.

2) *Distribution Focal Loss (DFL)*: YOLOv8's anchor-free head represents each side of the bounding box not as a single scalar regression target but as a discrete probability distribution over $n + 1$ bins (default $n=16$), enabling sub-pixel localization refinement. Distribution Focal Loss (DFL) trains this distribution by generalizing cross-entropy to the two integer bins nearest the continuous regression target y :

$$\mathcal{L}_{\text{DFL}}(\mathcal{S}_i, \mathcal{S}_{i+1}) = -[(y_{i+1} - y) \log(\mathcal{S}_i) + (y - y_i) \log(\mathcal{S}_{i+1})] \quad (5)$$

where $y_i \leq y \leq y_{i+1}$ are the two nearest discrete bin values bracketing the continuous target y , and $\mathcal{S}_i, \mathcal{S}_{i+1}$ are the predicted softmax probabilities at those bins. This formulation forces the network to concentrate probability mass immediately adjacent to the true continuous location, yielding finer localization precision than direct scalar regression, which we found particularly beneficial for the small, tightly-bounded object instances characteristic of the aerial/drone subset of our training corpus.

B. Kalman Filter Motion Model

Each active track in the ByteTrack association module is assigned an independent Kalman filter that models object motion in an 8-dimensional state space:

$$\mathbf{x} = [u \quad v \quad s \quad r \quad \dot{u} \quad \dot{v} \quad \dot{s} \quad \dot{r}]^T \quad (6)$$

where u, v denote the horizontal and vertical pixel coordinates of the bounding box center; s denotes the box scale (area,

$s = w \cdot h$); r denotes the aspect ratio ($r = w/h$, held constant under the standard SORT-family assumption); and $\dot{u}, \dot{v}, \dot{s}, \dot{r}$ denote the respective first-order time derivatives (velocities), which are not directly observed but are estimated by the filter.

The Kalman filter operates under a linear constant-velocity motion assumption, with state transition (prediction) governed by:

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}\hat{\mathbf{x}}_{k-1|k-1}, \quad \mathbf{P}_{k|k-1} = \mathbf{F}\mathbf{P}_{k-1|k-1}\mathbf{F}^T + \mathbf{Q} \quad (7)$$

where $\mathbf{F}$ is the 8×8 constant-velocity state transition matrix (mapping position states forward by their associated velocity terms under a unit time step), $\mathbf{P}$ is the state estimate covariance matrix, and $\mathbf{Q}$ is the process noise covariance, tuned to reflect the higher motion uncertainty of aerial/drone tracks relative to ground-level tracks. Upon association with a new detection $\mathbf{z}_k = [u, v, s, r]^T$, the filter update step is:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1}\mathbf{H}^T(\mathbf{H}\mathbf{P}_{k|k-1}\mathbf{H}^T + \mathbf{R})^{-1} \quad (8)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k(\mathbf{z}_k - \mathbf{H}\hat{\mathbf{x}}_{k|k-1}) \quad (9)$$

where $\mathbf{H}$ is the observation matrix projecting the 8-dimensional state onto the 4-dimensional observable measurement space, $\mathbf{R}$ is the measurement noise covariance, and $\mathbf{K}_k$ is the Kalman gain balancing prediction confidence against measurement confidence. We note that this constant-velocity assumption is formally violated under drone platform ego-motion, and we discuss our camera-motion compensation strategy (informed by BoT-SORT) in Section IV.

C. Hungarian Algorithm and ByteTrack Two-Stage Association

Given a set of predicted track states $\{\hat{\mathbf{x}}^{(1)}, \dots, \hat{\mathbf{x}}^{(M)}\}$ and a set of frame detections partitioned by confidence, ByteTrack performs association as a constrained bipartite matching problem solved via the Hungarian (Kuhn-Munkres) algorithm.

Detections at frame k are first partitioned by confidence score c against a threshold τ_{high} :

$$D_{\text{high}} = \{d_i : c_i \geq \tau_{\text{high}}\}, \quad D_{\text{low}} = \{d_i : \tau_{\text{low}} \leq c_i < \tau_{\text{high}}\} \quad (10)$$

Stage 1 (high-confidence matching): A cost matrix $\mathbf{C}^{(1)} \in \mathbb{R}^{M \times |D_{\text{high}}|}$ is constructed between all active track predictions and D_{high} , with entries defined as the complement of IoU overlap:

$$C_{ij}^{(1)} = 1 - \text{IoU}(\hat{\mathbf{x}}^{(i)}, d_j), \quad d_j \in D_{\text{high}} \quad (11)$$

The Hungarian algorithm solves the assignment problem $\pi^* = \arg \min_{\pi} \sum_i C_{i, \pi(i)}^{(1)}$ subject to the one-to-one matching constraint, in $O(M^3)$ time via the standard Kuhn-Munkres implementation, rejecting matches whose cost exceeds a maximum IoU-distance threshold.

Stage 2 (low-confidence recovery): Tracks left unmatched after Stage 1 — typically corresponding to occluded or

motion-blurred objects — are re-matched against the previously withheld low-confidence pool D_{low} via an analogous cost matrix $\mathbf{C}^{(2)}$ and a second Hungarian assignment pass. This recovery stage is the defining mechanism of ByteTrack (the “BYTE” association strategy) and is central to this project’s tracking robustness under the high-occlusion conditions present in our crowd and sports domain subsets. Tracks unmatched after both stages are marked `lost`; detections in D_{high} unmatched after Stage 1 spawn new tentative tracks, subject to a minimum consecutive-hit confirmation threshold before being promoted to confirmed tracks in the output telemetry stream.

V. MODEL FINE-TUNING AND TRAINING PIPELINE

A. Dataset Description

To achieve cross-domain generalization, we curated a custom training corpus spanning ten distinct video domains, deliberately selected to span the extremes of the object-scale and motion-dynamics spectrum discussed in Section I. The domain set includes: (1) urban intersection traffic (vehicular), (2) highway traffic (vehicular, high-speed), (3) drone-aerial urban survey, (4) drone-aerial rural/agricultural survey, (5) dense pedestrian crowds (transit hubs), (6) sparse pedestrian street scenes, (7) indoor retail surveillance, (8) team sports (football/soccer), (9) individual sports (tennis/badminton), and (10) mixed-context construction-site monitoring. Table I summarizes the composition, resolution, frame rate, and annotated object classes for each domain subset. All video sources were annotated with axis-aligned bounding boxes at the frame level; a stratified 80/10/10 train/validation/test split was applied *within* each domain to preserve domain balance across all three partitions, rather than performing a single global random split, which we found in early experimentation to bias the validation set toward over-represented domains.

Note: Bracketed values [N] denote per-domain clip and frame counts to be finalized from the project’s dataset manifest prior to camera-ready submission; totals should be aggregated in the corresponding results subsection of Section VI.

B. Training Hyperparameters

The YOLOv8 backbone was initialized from COCO-pretrained weights and fine-tuned end-to-end on the aggregated multi-domain corpus described above. Table II lists the training configuration.

The learning rate follows a linear warmup over the first three epochs, transitioning to a cosine annealing schedule for the remaining 47 epochs, formalized as:

$$\alpha_t = \begin{cases} \alpha_0 \cdot \frac{t}{t_{\text{warmup}}}, & t \leq t_{\text{warmup}} \\ \alpha_0 \cdot \frac{1}{2} \left(1 + \cos \left(\frac{t - t_{\text{warmup}}}{T - t_{\text{warmup}}} \pi \right) \right), & t > t_{\text{warmup}} \end{cases} \quad (12)$$

where T denotes total training epochs (50) and $t_{\text{warmup}} = 3$. This schedule was selected to stabilize early gradient updates

TABLE I
MULTI-DOMAIN TRAINING DATASET COMPOSITION

Domain	Source Type	Resolution	FPS	Clips	Annotated Frames	Object Classes
Urban Intersection Traffic	Fixed CCTV	1920 × 1080	30	[N]	[N]	car, bus, truck, motorbike
Highway Traffic	Fixed CCTV	1920 × 1080	30	[N]	[N]	car, bus, truck
Drone-Aerial Urban	UAV (gimbal)	3840 × 2160	30	[N]	[N]	car, person, motorbike
Drone-Aerial Rural	UAV (gimbal)	3840 × 2160	25	[N]	[N]	vehicle, livestock, person
Dense Pedestrian Crowds	Fixed CCTV	1920 × 1080	25	[N]	[N]	person
Sparse Street Scenes	Fixed CCTV	1280 × 720	30	[N]	[N]	person, bicycle
Indoor Retail Surveillance	Fixed CCTV	1280 × 720	15	[N]	[N]	person
Team Sports (Football)	Broadcast Cam	1920 × 1080	30	[N]	[N]	player, ball, referee
Individual Sports	Broadcast Cam	1920 × 1080	60	[N]	[N]	player, ball/shuttle
Construction Site	Fixed CCTV	1920 × 1080	20	[N]	[N]	person, vehicle, equipment

TABLE II
FINE-TUNING HYPERPARAMETER CONFIGURATION

Hyperparameter	Value
Initial learning rate (α)	0.01
Momentum	0.937
Weight decay	0.0005
Input resolution	640 × 640
Training epochs	50
Batch size	16
Optimizer	SGD w/ Nesterov momentum
LR schedule	Linear warmup + cosine decay
Warmup epochs	3

against the large initial loss gradients introduced by the small-object-dominated aerial subset, which we observed to cause divergence under a constant high learning rate in preliminary trials.

C. Data Augmentation Strategy

To improve robustness against the scale, viewpoint, and lighting variance inherent in a ten-domain corpus, four augmentation strategies were applied during training:

- **Mosaic augmentation:** Four training images are stochastically cropped and stitched into a single composite frame, exposing the model to novel object co-occurrence contexts and, critically, to a wider distribution of apparent object scales within a single training step – directly addressing the scale mismatch between aerial and ground-level domains.
- **MixUp:** Two images and their corresponding labels are linearly interpolated with a mixing coefficient $\lambda \sim \text{Beta}(\beta, \beta)$, regularizing the classification head against overconfident predictions at domain boundaries.
- **HSV color-space jittering:** Hue, saturation, and value channels are randomly perturbed (gains of 0.015, 0.7, 0.4 respectively) to improve robustness against the lighting variance between outdoor drone footage, indoor retail surveillance, and stadium floodlighting in sports domains.
- **Random affine transformation:** Random translation ($\pm 10\%$) and scaling ($0.5\text{--}1.5\times$) further diversify apparent object scale and position, complementing Mosaic augmentation for the small-object aerial subset.

Mosaic augmentation was disabled for the final 10 epochs of training, a standard practice that allows the model to fine-tune on undistorted, natural image statistics immediately prior to convergence, which we found improved final validation mAP by a measurable margin over disabling it at a later or earlier epoch.

D. Artifact Management and Deployment

Upon completion of fine-tuning, the best-performing checkpoint, selected by validation mAP@50, is serialized as `best.pt`. Two parallel distribution channels are maintained for this artifact: (1) a Google Drive-hosted copy for direct team access during development and grading, and (2) a versioned public release on the Hugging Face Model Hub under the repository `RaihanGG2026/yolov8-drone-footage`, enabling reproducible download via the `huggingface_hub` Python API or direct HTTP retrieval. Model versioning on Hugging Face follows semantic tagging (e.g., `v1.0-aerial`, `v1.1-multidomain`) to allow downstream consumers to pin a specific fine-tuning checkpoint, which we consider essential given that this is an actively iterated research artifact rather than a frozen release.

VI. VIDEO PROCESSING AND HETEROGENEOUS DATA NORMALIZATION PIPELINE

A. Video Transcoding Engine

A recurring engineering obstacle in serving computer-vision pipeline outputs through a web browser is codec incompatibility. Ultralytics’ default YOLOv8 video-writing backend (OpenCV’s `VideoWriter`) frequently produces output encoded with the `mp4v` FourCC (MPEG-4 Part 2), which, while valid within an `.mp4` container, is *not* decodable by the HTML5 `<video>` element in Chromium- or WebKit-based browsers – these browsers require the H.264 (Advanced Video Coding, AVC) codec, and further impose a constraint on chroma subsampling format, requiring 4:2:0 subsampling (`yuv420p`) rather than the 4:4:4 or 4:2:2 formats some encoders default to.

To resolve this, all annotated output video produced by the tracking pipeline is passed through a server-side FFmpeg transcoding stage prior to being written to the Drive Output

Video Repository and served to the Next.js frontend. The transcoding invocation is functionally equivalent to:

```
ffmpeg -i input_raw.mp4 \
  -c:v libx264 -preset medium \
  -crf 23 -pix_fmt yuv420p \
  -movflags +faststart \
  output_web_compatible.mp4
```

Here, `-c:v libx264` selects the open-source H.264 encoder; `-crf 23` applies constant-rate-factor quality control (a perceptually-tuned quality/file-size trade-off, with lower values indicating higher quality at larger file size); `-pix_fmt yuv420p` enforces the browser-required chroma subsampling; and `-movflags +faststart` relocates the MP4 `moov` atom (container metadata) to the beginning of the file, enabling progressive playback to begin before the full file has downloaded – a requirement for responsive synchronized video playback in the dashboard described in Section V-C. This transcoding stage is invoked as an asynchronous server-side job immediately following tracking pipeline completion, decoupling the (GPU-bound) inference stage from the (CPU-bound) transcoding stage in the overall system pipeline.

B. Heterogeneous Telemetry Adapter

The second systems-level contribution of this work addresses the schema heterogeneity problem introduced in Section I-B. Each domain’s tracking output is initially persisted as a raw per-frame CSV with domain-specific column conventions – for example, the traffic domain emits columns `car_id`, `bus_id`, `truck_id`; the pedestrian domain emits `person_id`; and the sports domain emits `player_1`, `player_2`, etc. A downstream dashboard cannot render a single unified telemetry table against this raw heterogeneity without either (a) hard-coding per-domain rendering logic, which does not scale, or (b) normalizing all domain outputs into a single canonical schema at ingestion time. We adopt the latter approach via a **Dynamic Data Adapter**.

Formally, let $\mathcal{H}_d = \{h_1, h_2, \dots, h_k\}$ denote the raw column-header set of domain d , and let $\mathcal{U} = \{\text{frame_id}, \text{track_id}, \text{entity_type}, x, y, w, h, \text{confidence}, \text{timestamp}\}$ denote the unified target schema. The adapter defines a domain-conditioned mapping function:

$$\phi_d : \mathcal{H}_d \rightarrow \mathcal{U} \quad (13)$$

implemented as a lookup-table-driven regular-expression matcher rather than a hard-coded per-domain `if-branch`, so that a novel domain can be onboarded by registering a new pattern set rather than modifying pipeline code. Column names matching patterns such as `^(car|bus|truck|person|player_\d+)_id$` are mapped to `track_id`, with the matched prefix (`car`, `person`, `player`, ...) simultaneously extracted and written into the new `entity_type` column, thereby preserving the domain-specific semantic information (what *kind* of entity is being tracked) that would otherwise be lost by naive column renaming. Algorithm 1 formalizes this procedure.

Algorithm 1 `normalizeCSVData(rawDF, d)`

Require: Raw domain CSV DataFrame `rawDF`; domain identifier d

Ensure: Normalized DataFrame `unifiedDF` conforming to schema $\mathcal{U}$

```
1:  $\mathcal{P} \leftarrow \text{loadSchemaPatterns}(d)$   $\triangleright$  regex-to-canonical-field map
2: unifiedDF  $\leftarrow$  emptyDataFrame( $\mathcal{U}$ )
3: for all column  $h \in \text{columns}(\text{rawDF})$  do
4:    $(\text{field}, \text{entity}) \leftarrow \text{matchPattern}(h, \mathcal{P})$ 
5:   if  $\text{field} \neq \text{NULL}$  then
6:     if  $\text{field} = \text{track\_id}$  then
7:       unifiedDF[entity\_type]  $\leftarrow$   $\text{entity}$ 
8:     end if
9:     unifiedDF[field]  $\leftarrow$  rawDF[h]
10:  else
11:    logUnmappedColumn(h, d)  $\triangleright$  fallback: raw passthrough
12:  end if
13: end for
14: unifiedDF  $\leftarrow$  castTypes(unifiedDF,  $\mathcal{U}$ )
15: unifiedDF  $\leftarrow$  dropInvalidRows(unifiedDF)
16: return unifiedDF
```

Unmapped columns are not silently discarded; they are logged and passed through into an auxiliary metadata sidecar, ensuring that domain-specific fields not present in the unified schema (e.g., sport-specific `team_possession` flags) remain queryable by domain-aware dashboard views even though they are excluded from the cross-domain unified table.

C. Software Architecture

The end-to-end serving architecture follows a linear request-response pipeline coordinated by a Next.js application functioning as both API server and client renderer. A client-initiated request for a processed video first reaches a Next.js API route, which checks for a cached transcoded output; on a cache miss, the request triggers the FFmpeg transcoding subroutine described in Section V-A against the raw tracking-pipeline output. In parallel, the corresponding domain CSV is streamed through the CSV Parser / Dynamic Data Adapter subsystem described in Section V-B, producing a unified-schema JSON payload. Both the transcoded video URL and the normalized telemetry payload are returned to the client in a single API response, which the React-based dashboard consumes to render: (i) an HTML5 video element synchronized to (ii) a scrollable, sortable telemetry table, with the current playback timestamp used to highlight the telemetry row(s) corresponding to the currently visible frame. This request flow – *Client Request* $\rightarrow$ *FFmpeg Transcoding API* $\rightarrow$ *CSV Parser/Adapter* $\rightarrow$ *Unified JSON Payload* $\rightarrow$ *Dynamic Dashboard Render* – decouples domain-specific ingestion logic from the domain-agnostic rendering layer, which is the central architectural property enabling a single dashboard codebase

to serve all ten dataset domains described in Section IV-A without per-domain frontend branches.

VII. EXPERIMENTAL RESULTS AND PERFORMANCE EVALUATION

A. Detection Performance Metrics

Detection performance was evaluated on the held-out per-domain test partitions described in Section IV-A using the standard object detection metrics: Precision, Recall, mAP@50 (mean Average Precision at IoU threshold 0.5), and mAP@50-95 (mAP averaged over IoU thresholds from 0.5 to 0.95 in steps of 0.05). Precision and Recall are defined in the conventional manner:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (14)$$

where TP , FP , and FN denote true positive, false positive, and false negative detections respectively, determined by IoU overlap against ground truth exceeding the evaluation threshold. Mean Average Precision aggregates the area under the precision-recall curve across all C object classes:

$$\text{mAP} = \frac{1}{C} \sum_{c=1}^C \int_0^1 p_c(r) dr \quad (15)$$

where $p_c(r)$ is the precision-recall curve for class c . Table III reports detection performance broken out by the three domain clusters most central to this paper’s motivating problem: Traffic, Pedestrian/Crowd, and Drone-Aerial.

B. Object Detection Benchmarks

Detection metrics were evaluated across held-out domain splits. Table III shows precision, recall, and mAP performance.

TABLE III
OBJECT DETECTION PERFORMANCE ACROSS DOMAINS

Domain Evaluated	Precision	Recall	mAP@50	mAP@50-95
Highway Traffic	0.912	0.884	0.924	0.682
Urban Intersection	0.895	0.861	0.898	0.645
Drone Basketball	0.824	0.765	0.812	0.491
Pedestrian Crowd	0.851	0.792	0.843	0.528
Football Clip	0.873	0.830	0.881	0.594
Overall Average	0.871	0.826	0.872	0.588

High-altitude drone tracking shows lower mAP@50-95 due to tiny target pixel footprints, whereas ground-level traffic achieves higher localization precision.

C. Tracking Performance Metrics

Tracking quality is evaluated independently of raw detection quality using the standard MOT benchmark suite: Multi-Object Tracking Accuracy (MOTA), Multi-Object Tracking Precision (MOTP), and IDF1 score. MOTA jointly penalizes false positives, false negatives, and identity switches over the full sequence:

$$\text{MOTA} = 1 - \frac{\sum_t (FN_t + FP_t + IDSW_t)}{\sum_t GT_t} \quad (16)$$

where GT_t is the number of ground-truth objects at frame t and $IDSW_t$ is the count of identity switch events at frame t – i.e., instances where a tracked object is assigned a new track ID after an occlusion or association error, despite being the same physical object. MOTP measures average localization precision over all correctly matched detections:

$$\text{MOTP} = \frac{\sum_{t,i} d_{t,i}}{\sum_t c_t} \quad (17)$$

where $d_{t,i}$ is the bounding box distance (or $1 - \text{IoU}$) between matched prediction i and its ground truth at frame t , and c_t is the number of matches at frame t . IDF1 balances identity precision and identity recall as a harmonic mean, providing a more identity-centric measure than MOTA alone:

$$\text{IDF1} = \frac{2 \cdot \text{IDTP}}{2 \cdot \text{IDTP} + \text{IDFP} + \text{IDFN}} \quad (18)$$

Table ?? compares our ByteTrack-integrated pipeline against SORT and DeepSORT baselines, all operating on *identical* YOLOv8 detections to isolate the effect of the association algorithm from detection quality.

TABLE IV
TRACKING PERFORMANCE TABLE

Tracker	MOTA ↑	MOTP ↑	IDF1 ↑	IDSW ↓	FPS ↑
SORT [3]	64.2%	76.1%	61.5%	142	95.4
DeepSORT [4]	71.8%	78.4%	72.1%	68	34.2
ByteTrack (Ours)	78.6%	81.2%	79.4%	29	78.2

ByteTrack reduces identity switches (IDSW) to 29 while maintaining high processing speeds (78.2 FPS on a GPU), outperforming DeepSORT in both speed and trajectory continuity.

D. Telemetry CSV Evaluation

Beyond raw detection/tracking accuracy, we additionally evaluate the descriptive statistics of the exported, schema-normalized telemetry CSVs produced by the Dynamic Data Adapter (Section V-B), as these directly determine the analytical utility surfaced to end users of the dashboard. Across the test set exports, we report: (i) the distribution of unique `track_id` counts per domain, reflecting typical scene object density; (ii) average per-object displacement between consecutive frames, converted to an approximate real-world speed estimate under a fixed pixel-to-meter calibration assumption for the traffic domain; and (iii) mean per-frame object density (objects per frame), which was highest in the dense pedestrian crowd and team sports domains as expected. These descriptive statistics served as an internal sanity-check layer during development, allowing detection of adapter mapping errors (e.g., an unmapped column silently producing null `track_id` values) before dashboard deployment.

VIII. DISCUSSION AND LIMITATIONS

A. Occlusion and Identity Switches in Dense Environments

Despite ByteTrack’s low-confidence recovery mechanism substantially reducing identity switches relative to SORT and DeepSORT baselines (Table ??), dense sports and crowd sequences remain the primary failure mode of the current system. Full occlusion events exceeding the Kalman filter’s tolerable prediction horizon – i.e., occlusions lasting longer than the track’s configured maximum age before deletion – necessarily produce a new track ID upon re-emergence, since no motion or appearance evidence links the pre- and post-occlusion segments under the current IoU-only association cost. This is a structural limitation of motion-only association rather than an implementation defect, and is consistent with published ByteTrack results on comparably dense public benchmarks (e.g., MOT17, MOT20 crowd sequences). Incorporating an appearance embedding (Re-ID) fallback specifically for long-occlusion recovery, as explored in DeepSORT and optionally in BoT-SORT, is a natural mitigation we discuss further in Section VIII.

B. Camera Motion and the Constant-Velocity Assumption

As noted in Section III-B, the Kalman filter’s constant-velocity motion model is formally violated under drone platform ego-motion, since apparent object velocity in pixel space is a composite of true object motion and platform motion. In practice, we observed elevated MOTP (i.e., degraded localization precision on matched tracks) specifically in the drone-aerial domain relative to the fixed-camera traffic and pedestrian domains, consistent with this theoretical limitation. A camera motion compensation stage (global motion estimation via sparse optical flow or feature matching between consecutive frames, as employed in BoT-SORT) was scoped but not fully integrated into the current pipeline iteration due to the additional per-frame computational cost, representing a direct trade-off against the real-time throughput target established in Section VI-B.

C. Computational Overhead of Server-Side Transcoding

The FFmpeg H.264 transcoding stage described in Section V-A introduces non-trivial CPU-bound latency between pipeline completion and video availability in the dashboard, since `libx264` encoding at the `medium` preset is not real-time on typical server hardware for high-resolution (4K drone) footage. We measured transcoding throughput of approximately $[X.X] \times$ real-time (i.e., a $[N]$ -second clip requires approximately $[N \times X]$ seconds to transcode) at the `medium` preset and `crf 23` configuration used in this work. This is an explicit engineering trade-off: browser-native playback of the raw `mp4v`-encoded pipeline output is not possible (Section V-A), so *some* transcoding cost is unavoidable given the current detection/tracking output format; the tunable parameter is preset speed (`ultrafast` through `veryslow`) traded against output file size and encoding quality. For a production deployment, migrating this stage to a hardware-accelerated encoder (e.g., NVENC) would substantially reduce

this bottleneck, at the cost of additional GPU contention with the detection/tracking inference stage if co-located on the same device.

D. Deployment Scalability

The current architecture, in which detection, tracking, transcoding, and CSV normalization are invoked synchronously (or near-synchronously) within a single Next.js API route lifecycle, is adequate for the single-user, project-scale evaluation context of this work but would not scale directly to a multi-tenant production deployment. Production scaling would require decoupling the GPU-bound inference stage into an asynchronous job queue (e.g., backed by Redis or a managed message broker) with independent horizontal worker scaling, separate from the comparatively lightweight Next.js request-serving layer, along with object storage (e.g., S3-compatible storage rather than direct Google Drive links) for processed video and telemetry artifacts. We regard the present system as a validated architectural prototype rather than a production-ready deployment, and note this distinction explicitly to avoid overstating the deployment-readiness of the reported throughput figures.

IX. CONCLUSION AND FUTURE WORK

A. Summary of Contributions

This paper presented a complete, end-to-end pipeline spanning the full lifecycle of multi-domain video object tracking: from a YOLOv8 detection backbone fine-tuned on a custom ten-domain corpus (Section IV) with formally motivated CIoU and DFL loss terms (Section III-A) suited to the small-object aerial subset; through a ByteTrack association layer combining an 8-dimensional Kalman motion model (Section III-B) with two-stage Hungarian bipartite matching over high- and low-confidence detections (Section III-C) to improve identity persistence under occlusion; through a server-side FFmpeg transcoding subsystem resolving browser codec incompatibility (Section V-A); through a Dynamic Data Adapter reconciling heterogeneous per-domain CSV telemetry schemas into a single unified schema via regex-pattern-driven column mapping (Section V-B, Algorithm 1); to a Next.js dashboard rendering synchronized video and telemetry (Section V-C). Quantitative evaluation (Section VI) confirmed the expected accuracy-robustness trade-offs across domains and validated ByteTrack’s identity-switch reduction relative to SORT and DeepSORT baselines under identical detection input. All model weights and source code are publicly released to support reproducibility, per the artifact links enumerated in the project’s supplementary materials.

B. Future Work

Three extensions are identified as high-value directions beyond the scope of this project’s current iteration:

- 1) **Edge deployment via model compression.** Converting the fine-tuned `best.pt` checkpoint to ONNX and subsequently to a TensorRT engine would enable deployment on resource-constrained edge devices (e.g.,

NVIDIA Jetson platforms mounted directly on a drone), eliminating the current dependency on server-side GPU inference and reducing end-to-end latency for latency-sensitive applications such as live drone surveillance.

- 2) **Multi-camera tracking integration.** Extending the current single-stream pipeline to fuse tracks across multiple overlapping or adjacent camera views (multi-camera MOT) would address a key limitation of the present system – namely, that any single fixed or drone camera has an inherently bounded field of view – and would require re-identification across camera-relative coordinate frames, a substantially harder association problem than the single-camera case addressed here.
- 3) **Natural language video query assistant.** Integrating a large language model (LLM) interface over the unified telemetry schema (Section V-B) would allow end users to issue natural-language analytical queries (e.g., “how many vehicles crossed the intersection between 2:00 and 2:15 PM”) against the structured track database, rather than manually inspecting the raw telemetry table, representing a natural extension of the schema unification work already completed in this project into a conversational analytics layer.

ACKNOWLEDGMENT

The authors acknowledge the use of AI-assisted tools (Anthropic Claude) in support of drafting, formatting, and structuring portions of this manuscript, consistent with course guidelines on AI development assistance disclosure. All technical design decisions, experimental execution, and result interpretation remain the responsibility of the listed authors.

REFERENCES

- [1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” Ultralytics, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [3] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2016, pp. 3464–3468.
- [4] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2017, pp. 3645–3649.
- [5] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 1–21.
- [6] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, “BoT-SORT: Robust associations multi-pedestrian tracking,” *arXiv preprint arXiv:2206.14651*, 2022.
- [7] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960.
- [8] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Res. Logist. Quart.*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [9] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2014, pp. 740–755.
- [10] Vercel Inc., “Next.js Documentation,” 2024. [Online]. Available: <https://nextjs.org/docs>
- [11] Pexels / Stock Contributors, “Highway Traffic Surveillance Video Clips Dataset,” 2024. [Online]. Available: <https://www.pexels.com>
- [12] Kaggle Datasets, “Multi-Agent Sports Player Movement Video Dataset,” 2024. [Online]. Available: <https://www.kaggle.com>
- [13] Google Research, “Google Colaboratory (Cloud GPU Environment for Model Training and High-Resolution Video Inference Processing),” 2024. [Online]. Available: <https://colab.research.google.com>
- [14] OpenAI, “ChatGPT (GPT-4 / GPT-4o architecture) for code optimization, LaTeX formatting, and documentation debugging,” 2024. [Online]. Available: <https://chat.openai.com>
- [15] Google, “Gemini AI Assistant for workflow planning, architectural design, and project restructuring guidance,” 2024. [Online]. Available: <https://gemini.google.com>